ASC – Materie Examen

Curs 1:

Sistem de calcul = ansamblu processor + memorie + periferice, toate interconectate intre ele

problemă principală - memory gap: până în anii 80, procesorul și memoria aveau latențe asemănătoare (load/store în memorie nu costa mai mult decât o adunare). de atunci, viteza de procesare a crescut exponențial, iar viteza de memorare a crescut liniar => avem nev. de soluții pentru a aduce datele în avans => cache, memorie virtuală

ILP = Instruction Level Parallelism = pot face mai multe instrucțiuni diferite în același timp pe același procesor (ex. 2 UAL, pot executa 2 operații de adunare simultan dacă instrucțiunile sunt independente)

SIMD -> aceeași instrucțiune, ILP -> instrucțiuni diferite

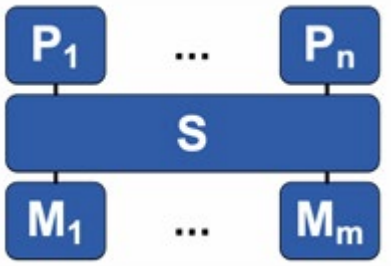
DLP = Data Level Parallelism (ne interesează mai mult la ASC) (ex. GPU pentru accerelări grafice, au căpătat mai multe putere procesoarele grafice)

Curs 2 – PMS:

- PMS = Procesor, Memorie și Switch
- Primitiva = (atribute, valori)
- **Memoria (M):**
  - In sist. de calcul, ne referim la **memoria RAM** = memoria care nu este persistentă după ce oprești calculatorul
  - Stochează și actualizează informația (date și instrucțiuni) prin operații de citire și de scriere
  - **necesită un mecanism de adresare pentru regăsirea datelor**
  - Ne intereseaza capacitatea, frecventa, latenta (= cat dureaza pana proceseaza o cerere de acces)
  - la calculatoare folosim **tehnologia dinamică (DRAM)**, nu folosim SRAM (static ram) deoarece costă mai mult. în afară de SRAM și DRAM, folosim **regiștri** – ca viteză, primii sunt regiștri, după care SRAM, după care DRAM
  - **Funcție**: primară / auxiliară
  - **Tehnologie**: bipolară / MOS / statică / dinamică
  - **Mod de acces**: aleator / secvențial / asociativ / FIFO / LIFO
  - **operații**: citire (memoria ROM) – citire/scriere (memoria RAM) -> ROM = Read-Only Memory = folosit pentru bootare
  - **lungimea cuvântului de memorie**: 8+1 / 16+2 / 32+4 / 64+8 biți
  - **capacitate**: 4MB / 16MB / ... / 2GB / 4GB / 16GB / 32GB
  - **ciclu de lucru**: 2ms / 4ms / ... / 100ns / 200ns
  - **Frecvența**: x Hz
- **Procesorul central (Pc):**
  - citește, interpretează și execută instrucțiuni mașină. Se ocupa de intreruperi, stie tot ce se intampla in sistemul de calcul
  - Inainte de 2000, conform legii lui Moore, frecventa unui procesor se dubla de la an la an, deci nu era nevoie de multicore. Dar : s-a atins limita tehnolgiei, nu mai poți mări frecvențele, se încălzesc prea tare circuitele și nu mai putem face overclocking -> motivul principal pentru aparitia **multicore**
  - **Tehnologii:** LSI | VLSI
  - **Funcții:** universale / specializate (ex. GPU)
  - **Formatul instructiunilor:** fix / variabil (CISC (arm) vs RISC (intel/amd: x86)
    - **RISC (Reduced Instruction Set Computer):** ARM, RISC-V, doar subsetul minimal de instrucțiuni necesar pentru a putea traduce orice program din limbaj de nivel înalt în limbaj de asamblare, are specifică execuția în pipeline
    - **CISC (Complex Instruction Set Computer):** Intel/AMD x86, conține și instrucțiuni compuse folosite frecvent (INC ECX), durata execuției instrucțiunilor poate varia foarte mult
  - **Procesor ILP (Instruction Level Parallelism)** -> procesor proiectat sa execute mai multe instructiuni dintr un singur flux de program in paralel, folosind **pipeline**
  - **Lungimea instrucțiunilor:** 8 / 16 / 32 / 64 / 128 biți
  - **Gradul de paralelism:** serial – single-core / paralel – multi-core
  - **Ciclu instrucțiune:** ciclu fix / ciclu variabil
  - **Dimensiunea registrelor interne:** 16 / 32 / 64 / 128 biți
  - **Numărul de core-uri:** 4 / 8 / 16
  - **Eterogeneitatea core-urilor**
- **Legatura (L) – Link:**
  - Asigură legătura fizică între diverse componente ale sistemului. Nu prelucrează informația. Este fie o magistrală, fie o interfață serială / paralelă
  - **Funcția:** legătura serială / paralelă / tip magistrală (se preferă în prezent legăturile seriale – USB, Ethernet – pentru că pot transfera mai multe date per unitate de timp, cu erori mai mici
  - **Lungimea cuvântului:** 5 / 6 / 7 / 8 / 16 / 32 / 64 + comanda
  - **Mod dialog**: sincron (trimit – ma blochez sa astept raspunsul – primesc raspunsul) / asincron (nu astept in mod activ raspunsul)
  - **Procesorul poate comunica cu periferice prin polling (sincron) sau prin sisteme de intrerupere (asincron)**

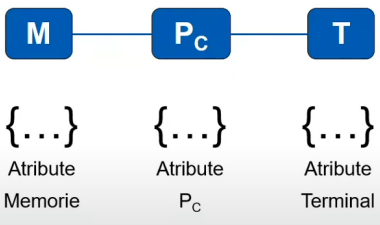
- **Mod de control al accesului:** înlănțuire serială (token) / interogare / cereri independente
- **Standard:** paralel / serial (RS232, SCSI, SATA, USB) / magistrală (MultiBus, MCI, HPIBus)

- **Switch/comutator (S)**



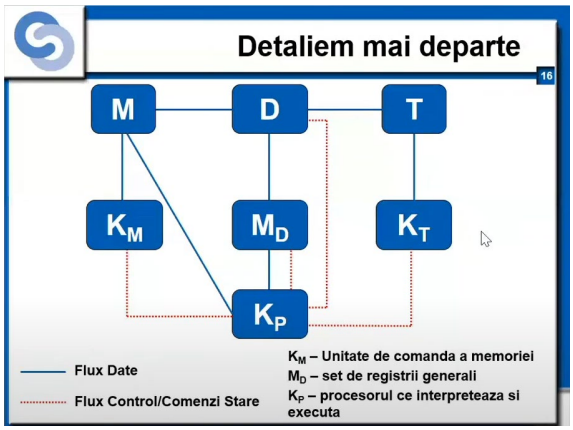
  - Face legătura între elementele sistemului, asigurând conexiuni între ele
  - Uneori procesorul vrea sa fie legat la un periferic, alteori la memorie
  - Cel mai simplu tip de comutator pt. P – M sau P - periferic: **buffer three-state** (conectat/deconectat/în aer)
  - Mai complex, spre. ex când un procesor vrea să acceseze memoria altui procesor -> vrem sa permutam comunicarile P – M (ex. P1 cu M3, P2 cu M4, P3 cu M1 etc.). Cel mai **complicat** switch este cel care face aceste permutari
  - **Structura:** ierarhica (leaga elemente de tipuri diferite) / neierarhica (cand elementele sunt de acelasi tip, ex. Core-urile unui procesor)
  - **Tip:** simplex/duplex/semiduplex (nu permit deodata sensurile contrare)
  - **Nr legaturi realizate** = nr intrari / iesiri
  - **Concurenta:** cate comunicatii permit simulan intre elementele de sus si elementele de jos
- **Unitatea de comanda (K)**
  - Componentă care exprimă controlul în diverse subansamble ale sistemului
  - Nu citeste & interpreteaza instructiuni aritmetice logice, ci doar comanda & control
  - Ex: K procesor pentru instruction decode, K memorie RAM pentru refresh
  - **Implementare**: conventionala / microprogramata
  - **Număr de stări:** n (2 stări = bistabil, 64 de stări = automat complex)
- **Procesor de I/E (P I/E)**
  - Este o primitiva ce poate fi incorporata in PC care are functie de I/E sau poate fi privita independent
  - **Functie:** DMA (= este o tehnică hardware prin care un periferic (sau un controler dedicat) transferă date direct către/-dinspre memoria principală **fără să implice continuu procesorul** în copierea fiecărui cuvânt de date.) / canal I/E / procesor specializat I/E
  - merge ca un fel de pipeline
  - Rata de transfer: Mb/s | Gb/s
  - **Mod de transfer: furt de ciclu** (folosește magistrala principală atunci când nu o folosește procesorul, dacă procesorul I/O are nevoie să treacă prin magistrala principală atunci o va face cu prioritate mai mică decât procesorul) / **rafală** (burst = odată ce cer 1 octet, dau și următorii octeți)
  - Lungimea cuvantului: 8/16 biti
- **Operatorul de date (D)**
  - **Preia datele analogice de la terminal si le transforma in digitale**
  - Microfon, camera video
- **Terminal (T)**

- Sa incercam o structura von Neumann



Procesor -> memorie -> periferice

!Memoria nu este cuplată tot timpul la magistrală, nici procesorul, ele se cuplează prin switch-uri doar când e nevoie

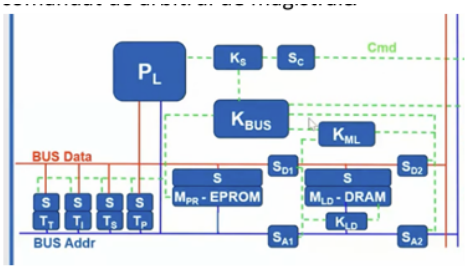


Subsistemul de I/O:

- **Asigura transferul datelor intre UC si mediul extern**
- **Transfer de date intre EP si UC:**
  - **Transfer programat**
  - **Acces direct la memorie (DMA – Direct Memory Access)**
  - **Canal de intrare/iesire**
  - **Calculator de intrare/iesire (Front End Computer/Processor - FEP)**

Single Board Computer:

- compus din procesor, memorie, interfețe I/O
- Pentru a gestiona accesul la magistrala (acces exclusiv) sunt necesare:
  - **KBUS: unitate de comanda pt accesul la magistrala (MAG)**
  - **KML: unitate de control al accesului la memoria locala**
  - KLD: unitate de comanda a memoriei locale (refresh/etc)
  - Ks: unitate de comanda a intregului sistem– asigura controlul procesoarelor la resursele sistemului



- PL = Procesor Local
- MLD = Memoria Locală
- liniile din dreapta = magistralele
- **KBUS = arbitru de magistrală principală**
- KLD = arbitru memoria locală
- pe magistrală pot scrie și periferice, KBUS se ocupă de acces

Curs 3:

Paralelismul este necesar deoarece nu mai este Legea lui Moore, adica frecventa nu mai creste de 2 ori mai rapid (mai degraba tranzistorii) la fiecare generatie, iar cerintele software sunt foarte mari, cum ar fi streaming 4k, jocuri cu cerinte foarte mari, etc.

Este ea mai usoara decat scrierea programelor seriale? NU. Scrierea paralela implica sincronizare, erori de cursa, nedeterminism (ordinea executarii variaza)

Paralelismul este necesar!

Nivele de paralelism:

- **La nivel de bit:** procesarea char vs short vs int vs long long vs long double etc.
- **La nivel de instructiune:** pipeline, VLIW (very long instruction word), hyperthreading, etc.
- **La nivel de memorie sau de I/O:** Suprapunerea operatiilor cu memoria/perifericele cu procesarea
- **La nivelul sistemului de operare:** Mai multe thread-uri sau procese care ruleaza in paralel
- **Distribuit:** Mai multe masini conectate impreuna, pentru a forma un supercalculator/cluster/cloud...

Roofline model:

Un model folosit pentru evaluarea performantei sistemelor de calcul per ansamblu (procesor + memorie + interconnect) vs necesitatile programelor.

Scop: sa arate rapid de ce nu obtii performanta teoretica de varf pe un CPU/GPU si unde merita sa concentrezi optimizarile

Aplicatiile SW au intensitate computationala diferita; intensitati computationale diferite au in general nevoi HW diferite (Definim **intensitatea computationala ca fiind numarul de operatii pe care le facem cu un byte din memorie odata ce l-am citit** (adus in procesor))

Taxonomia lui Flynn:

Impartirea sistemelor de calcul in functie de:

- Fluxul de Instructiuni– secventa de instructiuni executate de procesor
- Fluxul de Date– secventa de operanzi manipulata de procesor

Categorii:

1. SISD = Single Instruction, Single Data (structura Von Neumann)

Execută un singur fir de instrucțiuni pe un set de date la un moment dat – practic, procesorul secvențial tradițional. (ex. 8086, microcontrolere simple)

2. SIMD = Single Instruction, Multiple Data

Același cod (o instrucțiune vectorială) operează simultan pe mai multe valori. (ex. GPU)

3. MISD = Multiple Instruction Single Data

Mai multe unități rulează instrucțiuni diferite asupra aceluiași flux de date. Rar întâlnit în practică; se vede doar în sisteme de toleranță la erori, pipe-line securizat (ex. Sisteme de control aeronautic redundante)

4. MIMD = Multiple Instruction Multiple Data

Fiecare nucleu rulează propriul flux de instrucțiuni pe datele sale. Include aproape toate multicore-urile moderne și clusterele. (ex. Intel/AMD multi-core, servere ARM, clustere MPI)

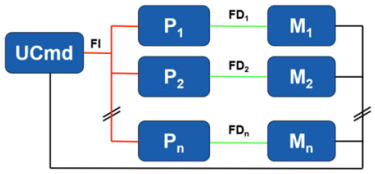
I – SISD (Single Instruction Single Data)

- **SISD = 1 FI & 1 FD**
  - Masina clasica, von Neumann (arhitectura unui singur core)
- **UCmd** = Unitate de comanda
- **P** = Unitate de prelucrare aritmetica
- **M** = Memorie
- FI = Flux Instructiuni
- FD = Flux Date



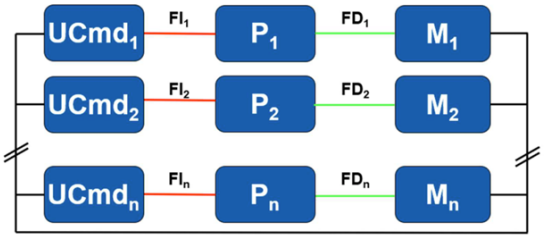
II – SIMD (Single Instruction Multiple Data)

- **SIMD = 1 FI & n FD**
  - Procesoarele de uz general au adesea si un subset de instructiuni vectoriale, SIMD, pe ~128 biti (deci nu un n f mare)
- SIMD sunt masini vectoriale:
  - O singura UCmd
  - Mai multe elemente de procesare (ALUs/branch exec unit/memory interact unit) si module de memorie (orice P vede orice M)
- Toate procesoarele fac aceeasi op impusa de UCmd prin FI



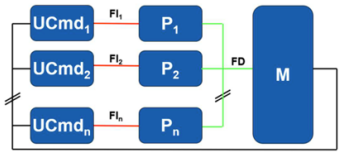
IV – MIMD (Multiple Instruction Multiple Data)

- **MIMD = n FI & n FD**
  - Toate supercalculatoarele din ziua de azi/clustere/cloud/grid ...
- MIMD pot comunica: (P-P sau P-M)
  - Nu intotdeauna orice processor poate accesa orice memorie
- Toate procesoarele participa la acelasi program



III – MISD (Multiple Instruction Single Data)

- **MISD = n FI & 1 FD**
  - Sisteme de calcul folosite de exemplu atunci cand o problema se poate rezolva in mai multe moduri, urmand apoi sa corelezi rezultatele
- MISD au:
  - Mai multe UCmd si procesoare
  - Un singur modul de memorie
- Domeniu de aplicatie restrans si special: aplicarea altor algoritmi pe aceleasi date (Apps: meteo/evidenta populatiei)



Arhitecturi SIMD vs MIMD:

GPU-urile sunt arhitecturi aflate la confluenta dintre principiile SIMD si MIMD

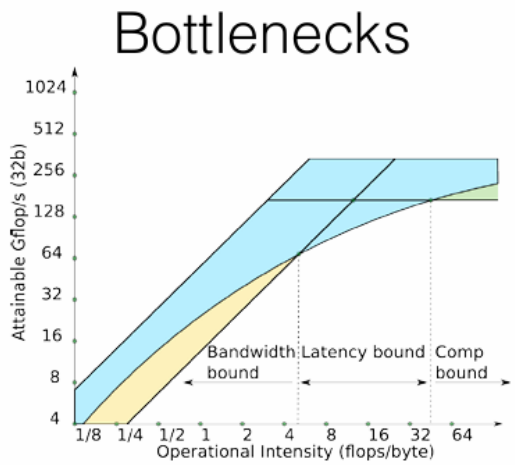
SIMD	MIMD
Sunt mai ieftine, pt ca au mai putine Ucmd	Sunt mai scumpe, mai complexe si mai diverse
Sunt mai putin flexibile ref. La alg. ce pot fi impl, din cauza necesitatii de sincronizare dupa fiecare instructiune	Sunt mai flexibile legat de algoritmii paraleli ce se pot implementa datorita posibilitatii de a se sincroniza oricand

Implementarea FORK & JOIN:

- Model clasic de implementare paralela („master-workers”): P1(master) pregateste taskurile si le pune in coada si P2...Pn inspecteaza coada si executa taskurile care asteapta. **Daca numarul de procesoare = numarul de procese → se executa simultan toate.**
  - **Pentru JOIN:** Pentru fiecare FI exista un contor unic ce e initializat cu 0 la inceput; Pentru fiecare FI contorul este incrementat si comparat cu n; Trebuie ca toate procesele sa se fi terminat (pana la n) si abia apoi se continua cu urmatoarea secventa liniara pe P 1 (master)

Curs 4:

Coalescing = Este o optimizare esențială pe GPU (și unele CPU-uri): – firele accesează date din memorie în ordine și contigu, astfel încât transferul să se facă eficient, în blocuri mari. – fără coalescing → fiecare fir accesează separat → zeci de transferuri mici, performanță foarte slabă.



Bottleneck-ul se schimbă în funcție de AI-ul programului tău:

- AI mic ⇒ memoria te limitează (trebuie să optimizezi accesul la date) - galben
- AI mediu ⇒ ai suficientă bandă, dar alte costuri interne domină (ex: latențe, cache misses) - albastru
- AI mare ⇒ ești limitat doar de cât de repede poți calcula (optimizezi FMA, SIMD, unrolling) – verde

Roofline model -> limitele depind de banda si de capacitatea de calcul

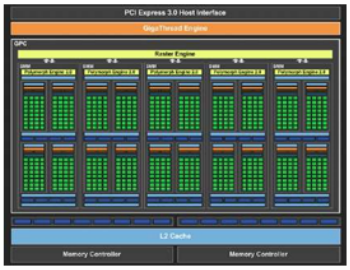
CPU -> putine thread uri, dar fiecare este puternic si bun la taskuri mari

GPU -> partile mult mai intense, mai multe fire -> fine-grained parallelism -> **ofera performanta mai buna si mai multa in paralelism masiv**

NVIDIA Kepler K40

- 2880 Streaming Processors/cores (SPs) organized as 15 Streaming Multiprocessors (SMs)
- Each SM contains 192 cores
- Memory size of the GPU system: 12 GB
- Clock speed of a core: 745 MHz





La procesoarele Pentium, marimea cache-ului a tot crescut, ceea ce este bine, deoarece acesta este foarte eficient, iar mărirea lui aduce beneficii directe de performanță.

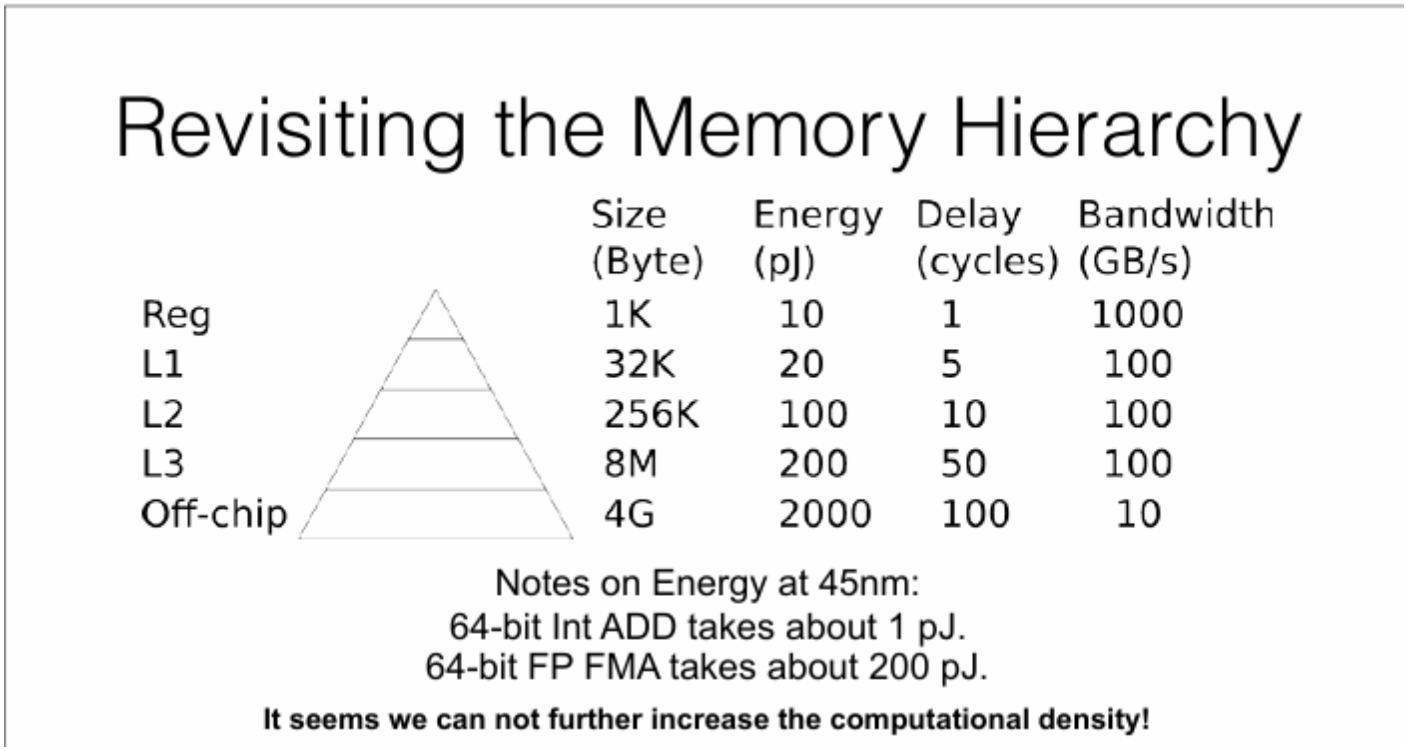
Cache:

- Este construit din SRAM (Static RAM), care e de ~100× mai rapidă decât DRAM-ul (memoria principală).
- E aproape de CPU, pe acelasi cip

Pentru singlecore, cache ul crestea pentru ca asa crestea si performanta. Cand nu s a mai putut, s a trecut la multicore, la care cache ul este in continuare important, fiind impartit pe nivele:

Nivel cache	Cine îl folosește	Caracteristici
L1 cache	✅ Doar un singur core (fiecare core are propriul L1)	Foarte mic (~32KB), foarte rapid, separat pe instrucțiuni și date
L2 cache	✅ De obicei, tot dedicat fiecărui core	Mai mare (~256KB–1MB), mai lent decât L1, dar tot rapid
L3 cache (sau LLC – Last Level Cache)	✅ Partajat între toate core-urile	Mare (până la zeci de MB), mai lent, dar esențial pentru cooperarea între core-uri
RAM (memoria principală)	✅ Toți core-urile o pot accesa (cu cost mare)	Dacă nu găsesc în cache, trebuie să acceseze DRAM

L1 și L2 = private per core, **L3 = cache partajat între toate core-urile**

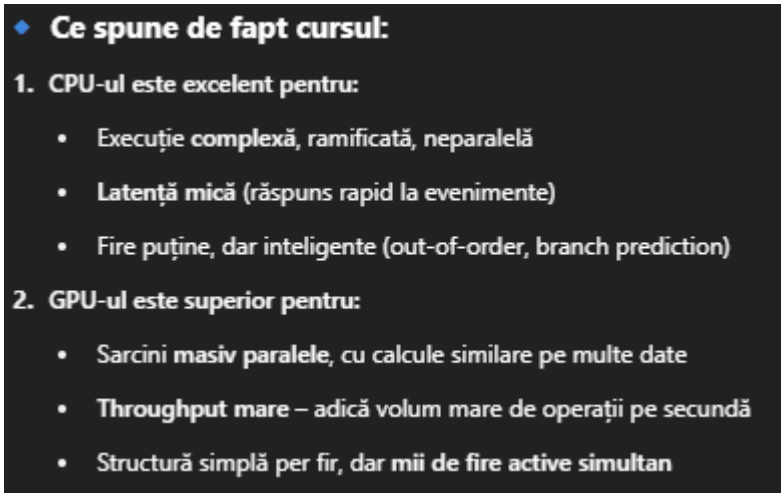


Ierarhia de memorie:

- **Registre**: Extrem de rapide, energie mică, viteză imensă – ideal să ții datele aici cât mai mult
- **L1 cache:** Foarte rapid, energie scăzută, bun pentru date accesate frecvent
- **L2/L3 cache:** Mai mari, dar mai lente și mai costisitoare energetic
- **Off-chip (RAM):** Foarte mare, dar de ~100x mai lents di costisitoate decat registrele

ILP măsoară câte instrucțiuni independente pot fi executate simultan de către procesor, într-un singur fir de execuție (adică fără multi-core, fără multi-threading).

Fine-grained interleaved threading este un model folosit în GPU-uri, unde execuția este simplificată, dar se bazează pe multe fire care rulează în paralel pentru a ascunde latențele în loc să le elimine cu execuție out-of-order. **-> pe scurt execută firele (threads) în funcție de disponibilitatea datelor și a resurselor, nu neapărat în ordinea strictă a codului global.**



SIMT (Single Instruction, Multiple Threads) -> este modelul folosit de GPU NVIDIA: **Toate firele (threads) execută aceeași instrucțiune, dar pe date diferite, fiecare având propriile registre și adrese.**

SM-ul (Streaming Multiprocessor) dintr-un GPU este un **core simplificat**, specializat pentru **execuție paralelă masivă** – nu pentru logică complexă sau control sofisticat.

Arhitecturi orientate pe throughput (in loc să te concentrezi pe a executa **rapid o singură sarcină (latență)**, te concentrezi pe a executa **foarte multe sarcini în paralel**):

- Fine-grained interleaved threading ~2x densitate de calcul
- SIMD/SIMT ~10x densitate de calcul
- Simple core ~2x

3 nivele de organizare a threadurilor: **GRID -> THREAD BLOCK -> THREAD**

! For CUDA, Hardware converts TLP into DLP at run time.

Thread blocks:

- Un array de thread uri concurente care executa acelasi program si pot coopera pentru a gasi rezultatul
- Este format din pana la 1024 threaduri
- Are forma si dimensiuni (1d, 2d, 3d) pentru threaduri
- A thread ID has corresponding 1d, 2d or 3d indices
- Maximum threads per SM: 2048

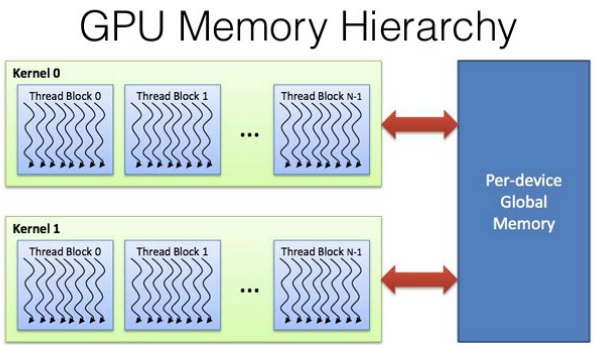
Pasi principali pentru programarea pe GPU:

- Inițializează datele pe CPU / sau alocă spațiu pe GPU
- Copiază datele de pe CPU pe GPU
- Calculează pe GPU
- Copiază datele înapoi de pe GPU pe CPU
  Reguli generale:
  - Minimizeaza transferul de date intre CPU si GPU
  - Maximizeaza numarul de fire pe GPU

Ierarhia de memorie – GPU:

- **Registrii -> ultra rapizi**
- **Memoria globala:**

- Este accesibila de catre toate firele care ruleaza pe GPU + de gazda (CPU)
- Durata de **viață a datelor** este de la **alocare până la dealocare**



Un kernel este un task pe care il are de facut GPU, iar acest task este impartit in mai multe threadblock uri, care la randul lor sunt impartite in mai multe threaduri

- **Memoria locala per thread:**
  - Fiecare thread are propria memorie locala
  - De obicei consta in registre (foarte rapide)
- **Memoria partajata per block:**
  - Memorie comuna intre toate thread urile dintr un block
- **Memoria constanta**

Există o ierarhie clară:

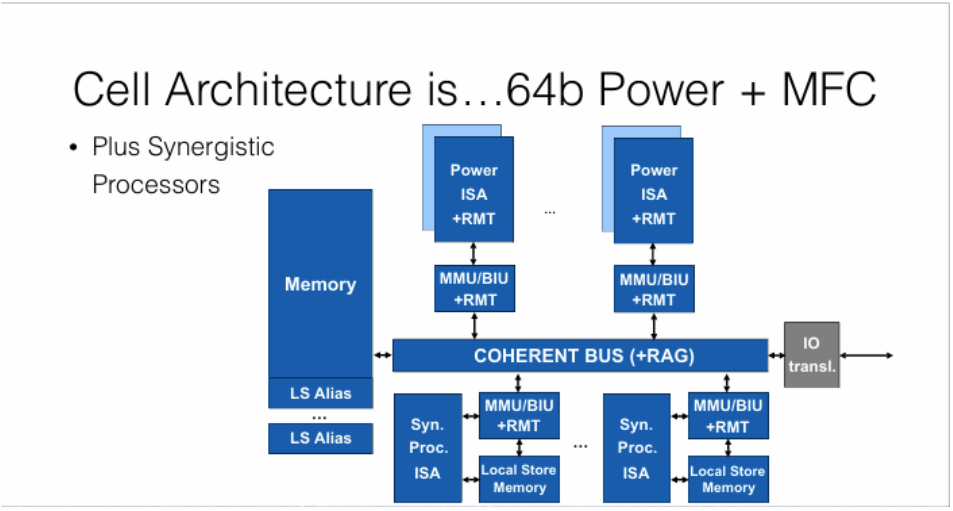
- **Thread** → registre, local memory
- **Block** → shared memory
- **Toate blocurile** → global memory & constant memory
- **Gazda (CPU-ul)** trebuie să transfere datele în GPU

GP-GPU(General-Purpose computing on Graphics Processing Units) = Folosirea GPU-urilor **nu pentru grafică**, ci pentru **calcule generale** (știință, AI, simulări, etc.)

Curs 5:

Arhitectura CELL:

1. **„Compatibility with 64b Power Architecture™”**
   ✅ În text: am menționat că **PPE-ul este un nucleu PowerPC pe 64 biți** – acesta rulează sistemul de operare și coordonează sarcinile.
2. **„Increased efficiency and performance”**
   ✅ Textul subliniază că arhitectura Cell depășește cele 3 „perete” clasice:
   - **Frequency Wall** → prin design neomogen și separarea logicii de control și calcule.
   - **Memory Wall** → prin **streaming DMA**, fără cache, datele sunt mutate manual.
   - **Power Wall** → multe nuclee simple + frecvență mare, dar tensiune mică.
3. **„Interface between user and networked world”**
   ✅ În text am menționat că PPE-ul rulează **sistemul de operare**, iar arhitectura permite:
   - **flexibilitate și securitate** (menționate și în slide)
   - **suport multi-OS**, inclusiv real-time (RTOS), și
   - **extensii pentru management în timp real** – esențial pentru aplicații embedded și multimedia.



Arhitectura de bază Cell

- **Este compatibilă cu PowerPC pe 64 biți (Power ISA)**
- Toate componentele sunt conectate printr-un **bus coerent**
- Fiecare nucleu are MMU/BIU (unitate de memorie + bus)
- Arhitectura păstrează coerența memoriei între nuclee (important pentru partajare de date)

Arhitectură extinsă – cu Memory Flow Control (MFC)

- Fiecare **SPE (nucleu auxiliar)** are:
  - propriul **Local Store (memorie locală)**, NU cache!
  - **MFC** = Memory Flow Controller, care controlează transferul de date prin DMA
- Toate componentele comunică tot prin **COHERENT BUS** (cu suport RAG – adresare globală unificată)
- **„LS Alias”** înseamnă că memoria locală poate fi „vizibilă” la adrese virtuale în sistem

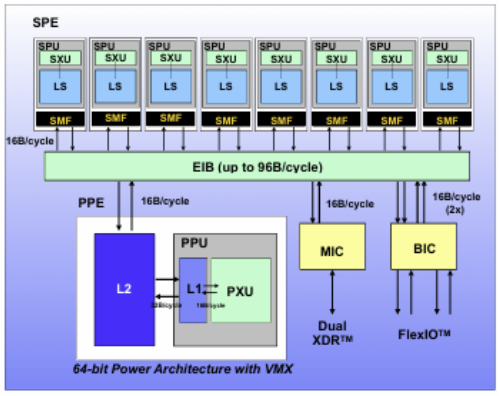
Cell BE vs. Pentium D

- Combines multiple high performance processors in one chip
  - 9 cores, 10 threads
  - A 64-bit Power Architecture™ core (PPE)
  - 8 Synergistic Processor Elements (SPEs) for data-intensive processing
- Current implementation — roughly 10 times the performance of Pentium for computational intensive tasks
  - Clock: 3.2 GHz (measured at >4GHz in lab)

	Cell	Pentium D
Peak I/O BW	75 GB/s	6.4 GB/s
Peak SP Performance	> 200 GFLOPS	> 30 GFLOPS
Area	235 mm^2	206 mm^2
Transistors	241 M	230 M

Cell BE Processor Features

- Heterogeneous multi-core system architecture
  - Power Processor Element for control tasks
  - Synergistic Processor Elements for data-intensive processing
- Synergistic Processor Element (SPE) consists of
  - Synergistic Processor Unit (SPU)
  - Synergistic Memory Flow Control
    - Data movement and synchronization
    - Interface to high-performance Element Interconnect Bus



PPE handles operating system and control tasks

SPE provides computational performance

CellBE Systems: game console systems, blades, some media servers, supercomputers. Real usage: Playstation 3

Pai ar trebui sa stii de cell ca sunt sisteme multi core eterogene, au arhitectura pe 64b si sunt ft bune dpdv al consumului de energie. Si sunt folosite in console de jocuri, in televizoare si in unele scopuri militare. Si in plus sunt mai bune fata de un procesor obisnuit.

Curs 6 – permutari

Permutari = moduri de interconectare intre resurse (precum switch uri sau procesoare). Dorim ca orice nod sa poata comunica eficient cu oricare altul. Modul in care conectam elementele determina o permutare a pozitiilor.

Permutarea de baza: - rearanjeaza in mod simplu conexiunile intre noduri (rotirea ciclica cu o pozitie la dreapta). Este de doua tipuri: **k inferior** – participa cei mai putini semnificativi k biti si **k superior** – participa cei mai semnificativi k biti

Permutarea cu intercalare perfecta: - rotirea ciclica cu o pozitie la stanga. Este tot de doua tipuri: **k inferior** – implica cei mai putini semnificativi k biti si **k superior** – implica cel mai semnificativi k biti

Permutarea cu negare de bit: - se neaga oricare dintre bitii reprezentarii binare a adresei

Permutarea fluture (butterfly): - interschimbarea bitului cel mai putin semnificativ cu bitul cel mai semnificativ. Este de doua tipuri: **k inferior** – implica numai cei mai putin semnificativi k biti si **k superior** – implica numai cei mai semnificativi k biti

Curs 6 – comutatoare (gestioneaza comunicarea intre procesoare, memorii si alte resurse hardware)

Sunt de doua tipuri:

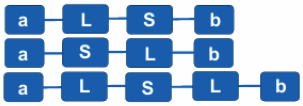
- **Ierarhice: conecteaza o componenta de tip a cu una de tip b (ex CPU - RAM)**
- **Neierarhice: conecteaza doua componenta de acelasi tip a (ex CPU - CPU)**

Comutatoare Ierarhice

- Comutatorul Simplu - Poarta
  - Asigura comunicarea intre componente de tip a si b
  - Este descris ca : S {poarta, 1a, 1b}



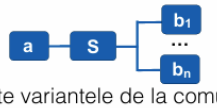
  - Variante posibile:
    - S {poarta, 1a, 1b, comutare la b}
    - S {poarta, 1a, 1b, comutare la a}
    - S {poarta, 1a, 1b, cu legatura la a si b}



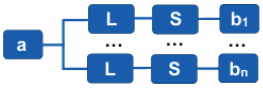
  - Poate conecta o resursa la magistrala sau o magistrala la alta magistrala

Comutatoare Ierarhice Duplex

- Asigura comunicatia intre o componenta de tip a si mai multe componente de tip b
  - Este descris ca: S {poarta, 1a, nb, concurenta 1, 1Sp}



  - Se pot considera toate variantele de la comutatoarele simple



Comutatoare Ierarhice Dual Duplex

- Asigura comunicatia intre doua componente de tip a si mai multe componente de tip b
  - Este descris ca: S {poarta, 2a, nb, concurenta 2, 2nSp}



  - Sunt posibile aceleasi variante ca la comutatoarele simple

Comutatoare Ierarhice Legatura Multipla

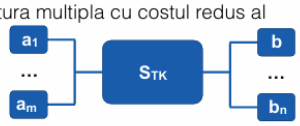
- Asigura comunicatia intre m componente de tip a si n componente de tip b
- Este cunoscut si ca Cross-Bar-Switch
  - S {CB, ma, nb, c = min(m, n), m*nSp}



- Orice a poate comunica cu orice b
- Pot avea loc transferuri multiple si simultane!
- Cel mai dificil este de implementat U_{Cmd} pentru S_{CB}
  - Trebuie sa rezolve toate posibilele conflicte

Comutatoare Ierarhice Trunchi K

- Imbina conectivitatea oferita de S legatura multipla cu costul redus al celorlalte tipuri de comutatoare
  - S {TK, ma, nb, c = k, k*(m+n)Sp}



- Orice a poate comunica cu orice b
- Nu totdeauna vei avea min(m, n) comunicatii simultane
  - Asa a aparut idea de a folosi k linii pentru comunicatie
- In functie de m si n si de trafic, se va determina un k optim
  - 8 procs si 4 memorii:
    - Pot fi maxim 4 accese la memorii → k=4
    - Nu are sens mai mult → k < n si k < m

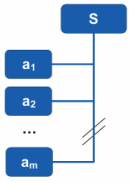
-> nu e nevoie de toate legaturile posibile daca nu sunt simultan atatea operatii ca la legatura multipla -> folosim doar k linii de comunicare (reduce costul)

Comutatoare neierarhice:

Tipuri:

Comutatoare Neierarhice Duplex

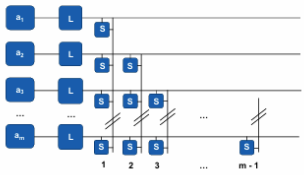
- Comutatoare Duplex Neierarhice
  - S{Dn; ma, c:1; mSp}



  - Bus Inlantuit

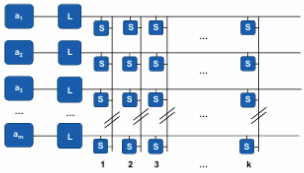
Comutatoare Neierarhice Legatura Multipla

- Legatura multipla – tip central
  - S{lm-neierarhica; ma, c:m/2; m(m-1)/2 Sp}



Comutatoare Neierarhice Trunchi-K

- Trunchi-K neierarhic
  - S{tk; ma, c:min(k,m/2); m*k Sp}



Fiecare nod este conectat direct la alte noduri prin **switch-uri (S)**.

Rețea **completă**: fiecare nod se poate conecta direct la toate celelalte.

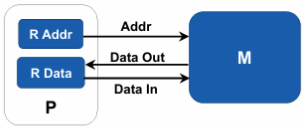
Numărul total de switch-uri este mare: **m(m–1)/2**, unde m = numărul de noduri.

Cost ridicat, **performanță excelentă**: latență mică, lățime de bandă maximă.

Este o rețea **tip fully-connected (clique)**.

Comutatoare Procesor Memorie

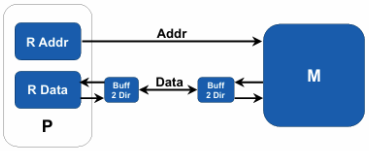
- Switch Processor Memory → S_{PM}
- Liniile de adrese si date sunt complet separate



R/W – Disjuncte → No Switch!
Nu se justifica ambele linii

Comutatoare Procesor Memorie

- Switch Processor Memory → S_{PM}
- D_{In} si D_{Out} sunt multiplexate prin magistrala de date



Bufferele sunt fie MUX fie TS

Comutatoare Procesor Memorie

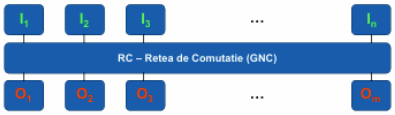
- Switch Processor Memory → S_{PM}
- Addr, D_{In} si D_{Out} sunt multiplexate prin aceeasi magistrala



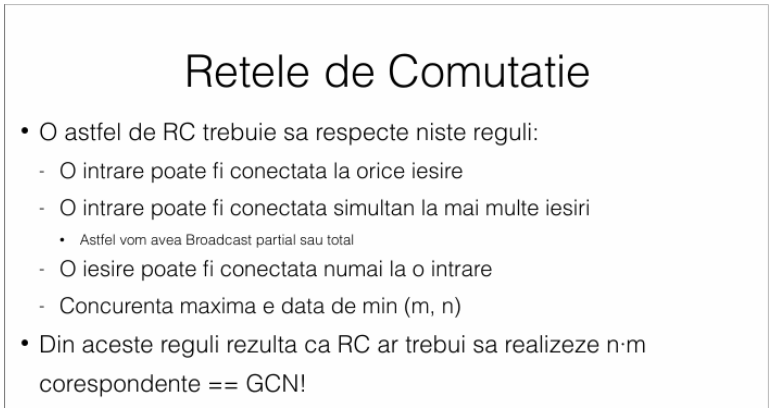
Bufferele sunt fie MUX fie TS

Interconectarea in Sistemele cu Resurse Multiple

- In structurile SIMD si MIMD, retelele de comutare (de interconectare a resurselor), au rolul cel mai important in asigurarea performantelor sistemului!
- Realizarea interconectarii resurselor (P, M & I/O) se face printr-o retea de comutatie (RC)



- I_i = Intrari & O_j = Iesiri; RC conecteaza n·m resurse
- Net (n, m) = Generalised Connection Network – graf in care arcele conecteaza perechi de resurse



Caracteristici GCN:

- **Intarzierea de propagare (Δt)** - timpul de raspuns ce depinde de numarul maxim de comutatoare elementare (de tip poarta) ce asigura legatura intre intrare si iesire
- **Complexitatea (nC)** – numarul total de comutatoare elementare utilizate in implementarea GCN-ului

Implementari ale RC:

- **Cu comutare directa:** crossbar complet – fiecare intrare are o conexiune posibila cu fiecare iesire (Avantaje: Δt = 1, totul se face intr un singur pas. Dezavantaje: greu de proiectat fizic, nu e scalabil). Ucmd controleaza toate switchurile
- **Implementare ierarhica:** In loc de retea completa, este o structura pe mai multe niveluri, mai usor de gestionat. Comutarea se face in mai multe etape -> creste timpul de comutare, dar este redusa complexitatea

Comutatoare elementare: au 2 intrari, 2 iesiri si 2 biti de comanda

Reteaua Crossbar: permite comunicarea directa intre oricare doua noduri, fara blocaje

Retele de comutare de tip Delta: sunt retele ierarhice formate din Crossbar uri si legaturi intermediare de intercalare perfecta (shuffle)

➔ Asigura conectivitate completa fara o retea completa fizic si ajuta fiecare intrare sa ajunga pas cu pas la orice iesire chiar si fara conexiuni directe

Retele bazate pe rutare: au ca nucleu legaturi directe intre o resursa si cele considerate vecine

Caracteristici: topologia (modul in care sunt interconectate nodurile), rutarea (calea de date pe care o urmeaza un mesaj) si comutarea (mecanismul ce determina cum si cand un canal de intrare este conectat la unul de iesire)

Topologii:

- Ortogonale (spatiu n dimensional)
- Strict ortogonale (o singura dimensiune, trecerea intr o dimensiune noua se poate face din orice nod)

Exemple: hipercub(2 cuburi unul intr-altul), Tor, Plasa N-diemensionala

Retele de tip Arbore: Exista mai multe cai posibile pentru mesaje – totdeauna se cauta drumul de lungime minima

Comportarea unui Procesor intr-un CB: se va folosi un **graf Markov.** Un procesor are 2 stari posibile: A = activ, W = wait

Comparatie Delta vs Crossbar:

CrossBar are cea mai bună performanță (cea mai mare P_a) – dar este foarte costisitor în implementare.

Delta este mai ieftină și mai scalabilă, dar performanța scade cu creșterea dimensiunii (multe conflicte).

- CrossBar = performanță mare, cost mare, implementare dificilă.
- Delta = performanță decentă, dar scalabilă și mai simplu de implementat.
- Probabilitatea P_a și rata r sunt esențiale în alegerea arhitecturii potrivite.

Curs 7 – Cache

Problema arhitecturilor moderne = memoria

The memory gap = diferenta imensa intre viteza de procesare si latenta memoriei (timpul de asteptare intre momentul cand procesorul cere date din memorie si momentul in care incepe sa le primeasca) – ex: din registru 0.25ns, din memorie 100ns, din harddisk 5ns

Avem nevoie de memoria cache din cauza memory gap. Acesta functioneaza folosind **localitatea datelor:**

- **Temporal locality:** daca ai accesat o data o variabila, probabil o vei accesa din nou curand (calculez suma elementelor dintr un vector)
- **Spatial locality:** daca ai accesat un element, probabil ii vei accesa si vecinii (elementele vectorului parcurse succesiv)

Design-ul cache-ului: make common case fast

Cache-ul este in general optimizat pentru CITIRE, pentru ca operatiile de citire sunt cam 20% din totalul de operatii. **Cache functioneaza eficient datorita TEMPORAL LOCALITY SI SPATIAL LOCALITY.**

CACHE HIT = datele cerute de procesor sunt deja in cache

CACHE MISS = datele cerute de procesor nu sunt inca in cache si trebuiesc aduse din memorie -> incetinire

Dimensiunea este seminificativ mai mica fata de memoria principala, dar este **organizat pe nivele, dimensiunea crescand dinspre L1->L3, dar viteza de acces fiind tot mai mica de la L1->L3.**

Linia de cache = cantitatea fixa care se aduce din memoria principala atunci cand avem cache miss -> contine mai multe elemente de control (bit de validitate, tag, indexul setului)

Cache set asociativ: considera bitii adresei de memorie impartita in TAG | INDEX | OFFSET.

3. Avand o adresa pe 64 biti, cine este TAG, INDEX, OFFSET?
   Fie adresa variabilei de accesat: 0x123456789abcdef0
   OFFSET = ultimii cei mai nesemnificativi 6 biti: 11 0000 (in binar)
   INDEX = urmatorii 9 biti imediat mai semnificativi decat OFFSET-ul: 101 1110 11 (in binar)
   TAG = restul de 49 biti mai semnificativi: 0001 0010 0011 0100 0101 0110 0111 1000 1001 1010 1011 1100 1 (in binar)

Scopul acestei împărțiri e să putem căuta rapid în cache:

- Cu **INDEX** intrăm direct în setul potrivit.
- Cu **TAG** verificăm dacă linia dorită este acolo.
- Cu **OFFSET** extragem byte-ul exact.

Tipuri speciale de cache:

- **Cache mapat direct: k = 1 (fiecare set are o singura linie)** – avantaje: usor de implementat, dezavantaje: sensibil daca 2 variabile au index identic
- **Cache total asociativ: k = tot cache-ul**

Politici de replacement: Least Frequently Used, Least Recently Used, First In First Out

Politici de scriere: RECALL, WRITE BACK (scriem doar in cache, nu si in RAM)**, WRITE THROUGH** (atunci cand scriem, scriem concomitent si in cache uri si in memoria principala)

THRASHING = doua sau mai multe adrese ajung sa foloseasca aceeasi linie de cache -> se scot reciproc din cache

Singurele doua instructiuni care au control explicit asupra cache-ului sunt PREFETCH (se poate specifica aducerea in avans in cache a unei variabile) si FLUSH (putem specifica inainte de un MISS de care variabile ne putem lipsi din cache)

Ierarhia de memorie in MIMD: Registre -> Cache de nivel 1,2,3, Memoria RAM a fiecarui nod si ansamblul memoriei RAM a tuturor celorlalte noduri

Unicul caz cand putem considera HDD ca fiind memorie: daca avem nevoie sa citim ceva dintr o pagina care nu este in RAM, vom citi de pe hard disk

TEOREMA CAP (pentru MIMD): Nu putem avea in acelasi timp 100% consistency, 100% availability, 100% partition tolerance

Curs 8 – SIMD:

- **arhitectură paralelă** folosită în prelucrări vectoriale.
- Toate procesoarele (P_1...P_n) execută **aceeași instrucțiune** pe **date diferite** (model paralelism de date).
- Eficientă în aplicații cu procesări masive de date: vectori, matrici, imagini etc.

Componente: Ucmd, Procesoare Pi, Memorii Mi, Switch P-P, P-M

 SIMD e foarte eficient în aplicații vectoriale.

 Este bazat pe o singură instrucțiune distribuită tuturor procesorarelor.

 Poate avea performanțe excelente **dacă datele sunt bine distribuite și accesate secvențial**.

 Nu este potrivit pentru aplicații care nu se bazează pe paralelism de date.

În SIMD există **un singur flux de instrucțiuni (FI)**, deci **nu pot fi două căi de execuție simultan** ca în procesoarele obișnuite (SISD). e aceea, **execuția condiționată** (if-else) se face **prin divizarea execuției în două faze**:

- Una pentru cazul în care **condiția e adevărată**.
- Alta pentru cazul în care **condiția e falsă**.

Se considera cate un bit de mascare asociat fiecarui procesor P_i, intr-un registru de masca (n biti): $P_i \leftrightarrow$ MASK[i]:

- MASK[i] = 1 → procesorul P_i functioneaza
- MASK[i] = 0 → procesorul P_i in asteptare & nu executa operatia curenta

Instructiuni: CLSS (comparare), CEQL (egalitate), BRALL (salt daca toti bitii din INDEX[i] = 1), BRNON (pentru INDEX[i] = 0), LDMSK, STMKS

Organizarea datelor:

Avem reprezentare directa pe linii sau pe coloane, dar nu este eficienta -> repartizarea deplasata ciclic (Datele sunt rearanjate astfel încât să se poată accesa **în paralel și linii, și coloane**. Se folosește **shuffling** (rețea de comutație) pentru a repoziționa datele.)

!SIMD foarte bun pentru operatii paralele pe date

Retelele de comutatie (RC) P-P sunt implementate la nivelul procesoarelor

Retelele de comutatie P-M pot fi implementate facil cu switch-uri Cross-Bar

Probleme de comunicatii intre procesoarele unui SIMD:

Distributia datelor, comunicare intre procesoare

Curs 9 – MIMD:

SIMD: Toate procesoarele execută aceeași instrucțiune, dar pe date diferite.

MIMD: Toate procesoarele participa la acelasi program (procesoare multiple, fiecare cu cod si date proprii)

Presiuni HW din partea SW

Problematica securitatii in MIMD:

- Distribuirea controlului este necesară din cauza plafonării frecvenței (Legea lui Moore). -> Asta aduce **probleme de autentificare și confidențialitate** – importante în aplicații critice (online banking, shopping etc.).

Sistemele suprasolicitate pot duce la **Denial of Service (DoS)**: Atacuri sau funcționare defectuoasă

Un serviciu **descentralizat** pe un sistem **MIMD** înseamnă că software-ul (SW) este scris în așa fel încât:

- **Să compenseze** pentru faptul că hardware-ul (HW) este format din multe componente separate (procesor + memorie diferite).
- Să facă **totul să pară unitar** (ca un singur sistem), chiar dacă în spate sunt multe noduri.

(ex. WWW, Google Drive, MS Teams)

Replicarea: Înseamnă să ai **mai multe copii** ale acelorași componente (CPU, memorie etc.) pentru a crește **disponibilitatea** sistemului. Motiv principal: **paralelizare** și **toleranță la defecte**.

Coerenta: Coerența înseamnă că **toate copiile datelor sunt la zi**. E greu de garantat 100% coerență în sisteme mari (ex: Big Data). -> Teorema CAP

Toleranta la defecte: -> Se referă la capacitatea unui sistem de a **continua funcționarea** chiar dacă unele componente eșuează. Se obține prin **redundanță** (copii suplimentare ale componentelor).

Scalabilitatea: -> Este **capacitatea sistemului** de a permite **adăugarea sau eliminarea de noduri** (procesoare, memorii etc.) **fără modificări majore** și fără a afecta performanța.

In sistemele MIMD scalabilitatea are legatura cu proprietatile comutatorului care interconecteaza nodurile individuale

- Comutator crossbar => scalabilitate mica (de obicei sistemele MIMD care comunica fizic prin shared memory)
- Retea de comutatie seriala => scalabilitate ridicata (de obicei sistemele MIMD care comunica fizic prin schimb de mesaje)

Echilibrarea incarcarii: se refera la evitarea solicitarii prea mari a unei componente a sistemului HW (sau SW), atata vreme cat alte componente sunt libere, sau mult mai putin stresate/incarcate

Problema accesarii eficiente a datelor in MIMD:

- **Modelele de acces la date in MIMD**
  - **Shared memory** (sistem multi-core/GPU cu memoria principala unica)
  - **Message passing** (sistem distribuit pe care rulam progr MPI)

Cand scriem SW pentru sisteme MIMD (sau SIMD ori GPU...) trebuie sa fim cu mare grija ca nu cumva 2 noduri/thread uri/procese diferite sa incerce simultan sa acceseze aceleasi date

Clasificarea structurilor:

Din punct de vedere arhitectural:

- **Sisteme multiprocesor**: sistem de calcul unitar unde o colectie de procesoare autonome functioneaza concurent si coopereaza la realizarea unei functii logice si de calcul de sub ctrl unui "executiv" unic -> comunica prin shared memory si se impart in **sisteme cu memorie distribuita** si **sisteme cu memorie partajata** (prin crossbar switch)
- **Sisteme multicalculator**: mai multe calculatoare autonome, interconectate, ce colaboreaza la realizarea unor task-uri din cadrul unei lucrari complexe -> comunica prin message passing si se impart in **multicalculator distribuite** si **multicalculator centralizate**

Din punct de vedere al accesului la memorie:

- **Sisteme cu memorie partajata (Shared Memory)**
  - **UMA = Uniform Memory Access** (orice proc se poate conecta uniform la un modul de memorie)
  - **NUMA = Non-Uniform Memory Access** (CSM, CIN, GSM, GIN)
  - **COMA = Cache-Only Memory Access** -> procesoarele au acces direct doar la cache -> nu au acces direct la propria zona de date sau la cea a celorlalte procesoare
- **Sisteme cu memorie distribuita (Message Passing)** -> multe noduri legate prin retea de comutatie

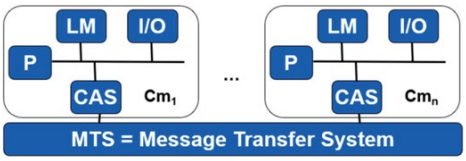
In sistemele MIMD procesoarele pot fi:

- Omogene -> procesoarele si elementele de I/O sunt identice
  - Cu procesoare simetrice – identice functional si I/O
  - Cu procesoare asimetrice – identice functional dar diferite ca I/O
- Eterogene

Sisteme strans cuplate vs slab cuplate:

Strans cuplate: procesoare conectate foarte eficient intre ele si la memoria partajata -> scalabilitate mica

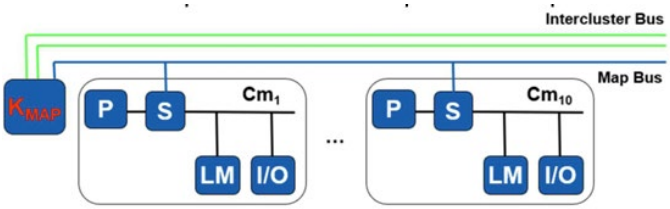
Slab cuplate: procesoare conectate mai putin eficient si nu au memorie partajata -> scalabilitate mare



(Avem CAS care functioneaza ca un semafor si el decine cine are voie sa trimita mesaje la MTS)

!!Ansamblul de core-uri CPU+GPU reprezinta un sistem mai slab cuplat decat ansamblul de core-uri CPU

Clusterul CM*:



Exemplu de arhitectura MIMD slab cuplata. Fiecare modul (Cm) este un calculator cu Procesor, memorie locala si I/O. Comunicarea între module se face printr-un **Map Bus**, gestionat de un controller numit **KMAP (un SO pe un SISD)**.

KMAP asigura:

- **Controlul in cadrul unui cluster – prin excluderea mutuala a accesului la MapBus**
- **Comunicarea intre clustere prin Intercluster Bus**

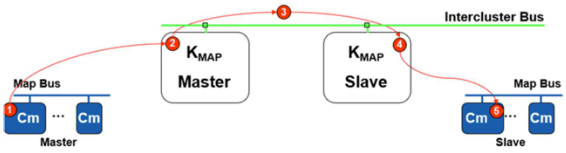
Functii de baza:

- **Mapare de adrese, comunicare, sincronizare**

Este compus din: Pmap (rol: rezolvare a adreselor virtuale in adrese fizice), Link, Kbus (arbitru de magistrala MapBus)

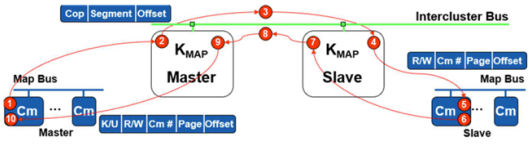
Comunicatia intre clustere (intercluster)

- 1. Cm Master trimite o cerere de transfer la K_{MAP} Master
- 2. K_{MAP} Master pregateste mesaj/pachet intercluster si codifica cererea
- 3. Mesajul intercluster e transmis pe Intercluster Bus pe baza algoritmilor de rutare
- 4. K_{MAP} Slave decodifica cererea primita si o trimite catre Clusterul sau local
- 5. Cererea de ciclu de memorie este trimisa la Cm Slave



Comunicatia intre clustere (intercluster) (2)

- 6. Cm Slave transmite rezultatul catre K_{MAP} Slave
- 7. K_{MAP} Slave pregateste mesajul intercluster (reactivarea contextului respectiv)
- 8. K_{MAP} Slave transmite rezultatul catre K_{MAP} Master
- 9. K_{MAP} Master receptioneaza si interpreteaza mesajul primit
- 10. Rezultatul este trimis catre Cm Master



Curs 10: Benchmarking

Nu era suficient doar sa citim specificatiile unor produse, ci a fost nevoie de crearea unor teste (benchmark-uri) care sa permita compararea performantelor intre sisteme/arhitecturi diverse

Tipuri de benchmarkuri:

- **Aplicatii reale**
- **Kernel**
- **Microbenchmark uri**
- **Sintetice – pe baza unor statistici**
- **Pentru I/O**
- **Pentru sisteme paralele**

Latency vs throughput:

Latency = timpul necesar terminarii unui task (timpul de executie)

Throughput = numarul de taskuri intr un timp fix (latimea de banda)

!Sistemele paralelele cresc de obicei **throughput-ul**, dar nu scad latenta.

Speedup = performanta

Calculul Performantei - Medii

- Media Aritmetica: $(1/N) * \sum_{P=1..N}$ Latenta(P)
  - Pentru marimi ce sunt proportionale cu timpul
- Media Armonica: $N / \sum_{P=1..N}$ 1/Throughput(P)
  - Pentru marimi ce sunt invers proportionale cu timpul
- Media Geometrica: $\sqrt[N]{\prod_{P=1..N}}$ Speedup(P)
  - Pentru marimi adimensionale

!!Build a balanced system -> use bandwidth to reduce latency

Benchmark-uri pentru Masini “Seriale”: **SPEC(Standard Performance Evaluation Corporation), GeekBench,**

Benchmark-uri pentru Masini “Paralele”: **LINPACK(cat de repede rezolva computer-ul un sistem dens de ecuatii liniare A·x=b de dimensiune n · n)**

Curs 11: Debugging, Profiling si Tracing

Pentru **debugging** facem atât analiza statica (review de cod de mana sau cu tool-uri) cât și analiza dinamica (rulare cu sau fără debugger)

Profiling = vedem performanta codului si unde ar putea fi el optimizat (**analiza performantei codului**)

Hotspots = functii care iau foarte mult din timpul total de executie

PAPI = un API de performanta care unifică și standardizează accesul la performance counters -> practic ofera o interfata pentru a masura performanta codului

Tracing = urmarirea in timp real a evenimentelor importante din rularea unui program.

Curs 12: Corectitudinea programelor paralele

Debugging-ul pt programele paralele este mai greu de facut; daca am rulat un program paralel o data si a functionat corect, **nu avem nicio garantie** ca el este fara bug-uri -> depinde de ordinea de planificare a thread urilor -> cele mai frecvente probleme: **race conditions** si **deadlocks**

Switch de context = daca un thread s a oprit, cand isi va relua activitatea, va continua fix de unde ramasese inainte

Întrebări examene

CA 2023 – Subiect rând par

1. Descrieți folosind primitivele PMS componenta Memorie cu minim 5 primitive (e.g. atribute/valori)

Memoria are rolul de a stoca și actualiza informația prin operații de scriere și de citire, necesitând un mecanism de adresare pentru a regăsi datele. Poate fi caracterizată în funcție de următoarele atribute: Funcția folosită (bipolară / MOS / statică / dinamică), modul de acces (aleator / secvențial / asociativ / FIFO / LIFO), tipul de operații suportate (citire / citire-scriere), capacitatea (între 4MB și 32GB) și frecvența.

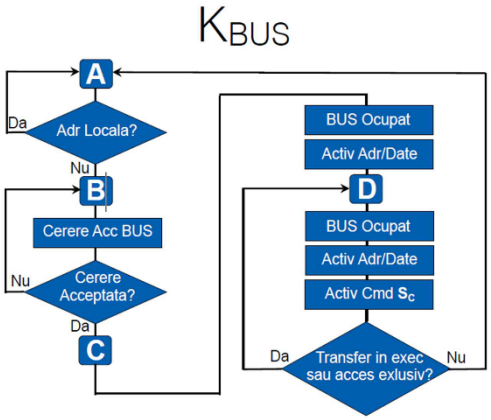
2. Descrieți comutatorul ierarhic de tip legătură multiplă (CB).

Acestea sunt cunoscute sub numele de Cross-Bar-Switch și au rolul de a asigura comunicația între M componente de tip A și N componente de tip B, unde orice componentă A poate comunica cu orice componentă de tip B. Pot avea loc transferuri multiple și simultane și este cea mai dificilă unitate de comandă de implementat, deoarece trebuie să se rezolve toate posibilele conflicte.

3. Explicați modelul Roofline ți modul în care acesta poate fi utilizat.

Modelul Roofline este folosit pentru evaluarea performanței sistemelor de calcul per ansamblu (procesor + memorie + interconectare) vs. necesitățile programelor. În practică, este folosit pentru a arăta rapid de ce nu este obținută performanța teoretică de vârf pe un CPU/GPU și unde merită să fie concentrate optimizările.

4. Realizați schema logică și descrieți funcționarea unității K_{BUS} din dispozitivele SBC.



K_{BUS} este unitatea de comandă care reprezintă procesorul local în relație cu magistrala externă. Este compusă din 4 stări. Starea A verifică dacă adresa este locală, dacă se adresează resurse locale, nu trebuie ieșit pe magistrală, dacă adresa nu este locală, se trece în starea B. Aceasta cere acces la magistrală și dacă este acceptată se trece în starea C. Aici, se emite Bus Busy (magistrala ocupată). Accesul pe BUS e exclusiv & cu rafală (overwrite); se rămâne în starea D pentru un nou ciclu dacă e cazul. După preluarea comenzii, procesorul face funcția de secvențiere a Adr, Date, Cmd, adică acțiunea corespunzătoare și în ordine cu aceste magistrale.

5. Descrieți o rețea de comutare de tip Delta cu 16 intrări si 25 de ieșiri și explicați modul de rutare corespunzător pentru ieșirea 6

Presupunem că avem 16 procesoare și 25 de memorii. $16 = 4^2$ și $25 = 5^2$, deci folosim 2 nivele de CB 4x5. Dacă vrem acces la memoria 6 (11 în baza 5), atunci se merge pe 1 (nivelul 0) și 1 (nivelul 1).

6. Sisteme cu memorie partajată: UMA și NUMA. Care sunt asemănările și care sunt deosebirile între aceste tipuri de sisteme? Oferiți exemple de astfel de sisteme.

Sistemele UMA (Uniform Memory Access) oferă accesul unic la date, orice procesor putând să se conecteze la un modul de memorie în mod uniform. Sistemele NUMA (Non-Uniform Memory Access) sunt mai complexe și presupun dependența accesului la memorie unde acesta trebuie efectuat. Ambele sunt arhitecturi cu memorie partajată, procesoarele pot accesa întreaga memorie disponibilă în sistem și se utilizează paralelismul la nivel de memorie. Un exemplu de sistem UMA este SMP (Systemic Multiprocessing), iar un exemplu de sistem NUMA este un supercalculator Cray.

7. Ce speedup maxim poate fi atins, cu un număr infinit de unități de procesare, dacă 85% din cod este paralelizabil?

Folosim legea lui Amdahl: $Speedup_{max} = \frac{1}{S+\frac{P}{N}}$, unde S = porțiunea secvențială din cod, P = porțiunea paralelizabilă, N = numărul de unități de procesare. Aplicând formula obținem $\frac{1}{0.15+\frac{0.85}{\infty}} \approx 6.67$, deci un speedup maxim aproximativ de 6.67.

8. Ce este un Benchmark? Oferiți o taxonomie a benchmark-urilor. Oferiți detalii pentru cel puțin două benchmark-uri pentru sisteme de calcul cu un singur CPU.

Un benchmark este un test standardizat pentru a măsura și compara performanța unui sistem de calcul (hardware și/sau software). Acestea pot fi clasificate după nivelul de testare (aplicații reale, kernel, micro-benchmark-uri, sintetice, pentru sisteme I/O, pentru sisteme paralele). Două benchmark-uri pentru sisteme de calcul cu un singur CPU sunt: Dhyrstone – benchmark sintetic, folosit pentru a măsura performanța sistemelor integer (non-floating-point), care sintetizează operații frecvente în aplicații reale (apeluri de funcții, atribuiri, structuri de control); și Whetstone – benchmark axat pe operații cu virgulă mobilă, care include funcții matematice, operații pe vectori.

9. Ce înseamnă stress testing? La ce este folosit? Exemplificați printr-un exemplu.

Stress testing este o tehnică de testare a unui sistem sau program prin rularea sa sub condiții extreme de sarcină sau resurse solicitate, pentru a observa cum se comportă în situații de suprasolicitare. El verifică cât de robust și stabil este un sistem, identifică bottleneck-uri și posibile crash-uri și evaluează capacitatea de recuperare a sistemului. Un exemplu poate fi reprezentat de generarea de 10000 de cereri simultane pe secundă pe un server web (mult peste nivelul normal).

10. Numiți cel puțin 3 caracteristici distinctive ale ultimelor arhitecturi de procesoare Nvidia/AMD/Intel (2020+) pe care le-ați folosit.

Procesoarele moderne integrează arhitecturi multi-core hibride sau masiv paralele, acceleratori hardware specializați pentru AI, grafică sau alte domenii nișate și interconectări rapide și memorii de ultimă generație pentru a susține sarcini variate, de la gaming, la cloud computing și deep learning.

CA 2023 – Subiect rând impar

1. Descrieți folosind primitivele PMS componenta Unitatea de comandă cu minim 5 primitive (e.g. atribute/valori)

Unitatea de comandă este componenta care exprimă controlul în diverse subansamble ale sistemului. Are o funcție de comandă a resurselor asociate unităților funcționale. Ea nu citește sau interpretează instrucțiuni aritmetice logice, ci doar comandă și controlează. Poate fi caracterizată în funcție de următoarele atribute: funcția (ce resursă comandă), implementarea (convențională / microprogramată) și numărul de stări (descrie complexitatea automatului).

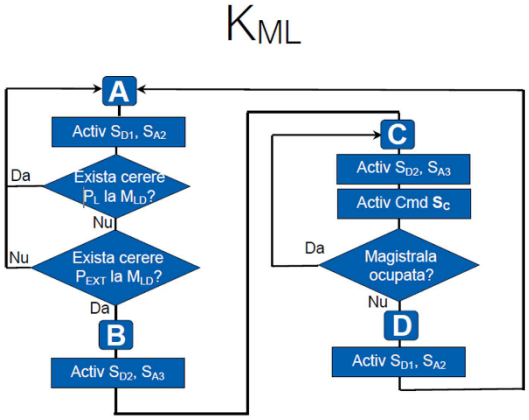
2. Descrieți comutatorul ierarhic de tip trunchi K.

Comutatorul trunchi K îmbină conectivitatea oferită de Cross-Bar-Switch cu costul redus al celorlalte tipuri de comutatoare. Într-o configurație cu M componente de tip A și N componente de tip B, orice componentă A poate comunica cu orice componentă B, însă nu întotdeauna vor fi min(M, N) comunicații simultane. În funcție de M, N și trafic, se va determina un număr de comunicații optim.

3. Ce reprezintă intensitatea computațională / aritmetică și cum se leagă ea de modelul Roofline?

Intensitatea computațională este definită ca numărul de operații aritmetice realizate cu un singur byte de memorie adus în procesor. În modelul Roofline, această intensitate este folosită pentru a determina dacă performanța unei aplicații este limitată de puterea de calcul a procesorului (compute-bound) sau de lățimea de bandă a memoriei (memory-bound). Astfel, modelul Roofline ajută la identificarea principalului factor care limitează performanța și oferă direcții clare pentru optimizare.

4. Realizați schema logică și descrieți funcționarea unității K_{ML} din dispozitivele SBC.



K_{ML} este un automat finit care asigură gestionarea eficientă a cererilor și resurselor, evitând conflictele pe magistrală. Este compus din 4 stări. Starea A verifică dacă există cerere de la procesorul extern (și nu de la cel intern) pentru anumite acțiuni și trece în starea B, unde activează alte acțiuni ți trece în starea C. Acolo, verifică dacă magistrala este ocupată și, în caz contrar, execută acțiunile inițiale.

5. Descrieți o rețea de comutare de tip Delta cu 25 de intrări si 16 ieșiri și explicați modul de rutare corespunzător pentru ieșirea 9.

Presupunem că avem 25 de procesoare și 16 memorii. $25 = 5^2$ și $16 = 4^2$, deci folosim 2 nivele de CB 5x4. Dacă vrem acces la memoria 9 (21 în baza 4), atunci se merge pe 2 (nivelul 0) și 1 (nivelul 1).

6. Sisteme cu memorie partajată: CC-NUMA și COMA. Care sunt asemănările și care sunt deosebirile între aceste tipuri de sisteme? Oferiți exemple de astfel de sisteme.

Sistemele CC-NUMA (Cache Coherent NUMA) și COMA (Cache Only Memory Access) se aseamănă prin dependența timpului de acces la memorie în funcție de distanța fizică între procesor și blocul de memorie, optimizarea accesului la date și minimizarea latențelor prin folosirea cache-urilor locale și folosirea în arhitecturi scalabile. Cele două sisteme se deosebesc prin coerența cache-urilor (asigurată prin protocoale hardware la CC-NUMA și gestionarea datelor prin migrare și replicare la COMA) și locația datelor (locație fixă în memoria fizică la CC-NUMA și migrarea între noduri în funcție de acces la COMA). Un exemplu de sistem CC-NUMA sunt serverele multi-socket moderne, iar un exemplu de sistem COMA este DDM (Data Diffusion Machine).

7. Ce speedup maxim poate fi atins, cu un număr infinit de unități de procesare, dacă 70% din cod este paralelizabil?

Folosim legea lui Amdahl: $Speedup_{max} = \frac{1}{S+\frac{P}{N}}$, unde S = porțiunea secvențială din cod, P = porțiunea paralelizabilă, N = numărul de unități de procesare. Aplicând formula obținem $\frac{1}{0.30+\frac{0.70}{\infty}} \approx 3.33$, deci un speedup maxim aproximativ de 3.33.

8. Descrieți pe scurt benchmark-ul LINPACK, Prezentați avantaje și dezavantaje ale acestuia.

Benchmark-ul LINPACK este folosit pentru sistemele paralele și măsoară cât de repede rezolvă computerul un sistem dens de ecuații liniare Ax=b de dimensiune nxn. Avantajele acestuia sunt că este ușor de definit și clasificat pe baza lui și că permite modificarea dimensiunii problemei, iar dezavantajele lui sunt că scoate în evidență doar viteza peak și numărul de procesoare și nu evidențiază explicit rata de transfer locală și rețeaua de comunicare.

9. Ce înseamnă random scheduling și ce limitări are această tehnica?

Random scheduling este o tehnică de planificare a proceselor în care task-ul următor care va fi executat este ales aleatoriu, fără o ordine prestabilită sau criterii de priorități. Limitările includ lipsa predictibilității, ineficiență pentru procese cu priorități diferite sau posibilitatea unui proces de a intra în starvation pentru că nu a fost ales de mult timp.

10. Care este diferența dintre memoria partajată (shared) și cea globală în sistemele de calcul de tip accelerator matematic dezvoltate de Nvidia, AMD sau Intel?

Memoria globală este accesibilă tuturor thread-urilor, are o capacitate mare, dar și o latență mare și este tipic utilizată pentru stocarea persistentă a datelor mari. În schimb, memoria partajată este accesibilă numai thread-urilor din același bloc, este mai mică, dar și cu o latență mai mică și permite comunicarea rapidă între procese și stocarea temporară a datelor.

CB 2023 – Subiect rând par

1. Descrieți folosind primitivele PMS componenta Procesor de Intrare-Ieșire cu minim 5 primitive (e.g. atribute/valori)

Procesorul de Intrare-Ieșire este o primitivă care poate fi incorporată în procesorul central sau poate fi privit ca fiind de sine stătător. Acesta este caracterizat de următoarele atribute: funcția (DMA / canal I/E / procesor I/E), rata de transfer (GB/s / MB/s), modul de transfer (furt de ciclu / rafală) și lungimea cuvântului (8 / 16 biți).

2. Descrieți comutatorul ierarhic de tip legătură multiplă (CB).

Acestea sunt cunoscute sub numele de Cross-Bar-Switch și au rolul de a asigura comunicația între M componente de tip A și N componente de tip B, unde orice componentă A poate comunica cu orice componentă de tip B. Pot avea loc transferuri multiple și simultane și este cea mai dificilă unitate de comandă de implementat, deoarece trebuie să se rezolve toate posibilele conflicte.

3. Comparați permutarea Bit Reversal Permutation (negare de bit) și permutarea Butterfly Permutation (fluture).

Permutarea cu negare de bit (Bit Reversal Permutation) și Permutarea fluture (Butterfly Permutation) sunt două tipuri de permutări elementare utilizate în organizarea datelor și interconectarea resurselor în arhitectura sistemelor de calcul. Principala diferență constă în maniera rearanjării biților: permutarea cu negare de bit rearanjează pozițiile într-o manieră statică, prin oglindire, în timp ce permutarea fluture definește modul de conectare și procesare în mai multe etape. Prima este o permutare globală, iar cea de-a doua este o permutare etapizată.

4. Exemplificați modul de funcționare al DMA în sistemele cu procesoare de tip general CPU.

DMA permite transferul direct de date între dispozitivele periferice și memoria RAM, fără implicarea CPU, eliberând-ul pentru alte sarcini și crescând eficiența sistemului. CPU configurează controlul DMA prin adresa sursă, adresa destinație și dimensiunea transferului, după care controller-ul DMA preia controlul asupra magistralei de sistem și realizează transferul de date. La final trimite un interrupt către CPU când transferul este complet.

5. Tradeoff-uri de utilizare a rețelelor de comutare de tip Delta, Cross-Bar sau trunchi K.

Rețelele de comutare de tip Delta au scalabilitate bună, cost redus și flexibilitate, dar au și un număr mare de niveluri care cresc latența, pot bloca unele căi chiar dacă există resurse disponibile și au o complexitate mare de rutare. Rețelele CB eu latență minimă, nu au blocări interne și sunt simplu de implementat, în schimb au un cost ridicat, un consum mare de putere și o scalare limitată. Rețelele trunchi K au scalabilitate foarte bună, latență echilibrată și toleranță la blocare, dar prezintă complexitate ridicată, cost moderat și overhead de control.

6. Cum sunt sistemele de virtualizare din ziua de azi influențate de arhitecturile de tip COMA?

Arhitecturile COMA (Cache-Only Memory Architecture), prin conceptul lor de memorie distribuită și migratoare, au influențat sistemele moderne de virtualizare prin inspirarea tehnicilor de migrare a paginilor de memorie între noduri și mecanisme avansate de consistență a datelor. Aceste principii permit optimizarea accesului la memorie în medii virtualizate distribuite, reducând latențele și îmbunătățind scalabilitatea sistemelor. Astfel, virtualizarea modernă beneficiază de gestionarea dinamică a memoriei și sincronizarea eficientă între mașini virtuale, facilitând performanța și flexibilitatea în cloud și data center.

7. Cât din cod trebuie să fie paralelizabil pentru ca speedup-ul obținut să fie 1000?

Folosim legea lui Amdahl: $Speedup_{max} = \frac{1}{S+\frac{P}{N}}$, unde S = porțiunea secvențială din cod, P = porțiunea paralelizabilă, N = numărul de unități de procesare. Presupunem un număr infinit de procesoare. Aplicând formula obținem $1000 = \frac{1}{(1-P)+\frac{P}{\infty}} \rightarrow P = 99.9\%$, deci 99.9% din cod ar trebui să fie paralelizabil.

8. Care este ciclul tipic de dezvoltare al unui program cu mai multe thread-uri pe arhitecturi de tip GPU?

Ciclul tipic de dezvoltare a unui program cu mai multe thread-uri pe arhitecturi GPU include următorii pași: mai întâi, se identifică porțiunile de cod care pot fi paralelizate și se concepe un kernel, adică o funcție care va fi executată concurent de multiple thread-uri; apoi se alocă și se transferă datele necesare între memoria CPU (host) și memoria GPU (device); după aceea, kernel-ul este lansat pe GPU, specificând numărul de thread-uri și structura blocurilor pentru a asigura o paralelizare eficientă; thread-urile din GPU execută kernel-ul concurent, accesând memoria globală sau partajată după caz; la final, rezultatele sunt copiate înapoi de pe GPU în memoria CPU pentru a fi utilizate în continuare. Pe parcurs, programatorul trebuie să gestioneze sincronizările între thread-uri și să optimizeze accesul la memorie pentru performanță maximă.

9. De ce este importantă alegerea workload-ului de test pentru a verifica corectitudinea și eficiența unui program?

Un workload balansat acoperă toate cazurile relevante de utilizare, permite detectarea bug-urilor, erorilor de calcul, condițiilor de cursă sau a deadlock-urilor. Eficiența unui program variază în funcție de tipul și volumul datelor procesate, deci un workload realist reflectă comportamentul programului în condiții de utilizare normale sau intensive. Un workload nerelevant poate duce la optimizări greșite sau concluzii false privind performanța. Un exemplu concret este aplicarea unui algoritm de sortare pe un vector deja sortat (cazul ideal), care poate părea eficient, dar se poate dovedi ineficient pe un vector mare și aleator (cazul real).

10. Cum e diferită o aplicație CPU intensivă de un yield sau wait (random)?

O aplicație CPU intensivă folosește în mod constant unitatea de procesare pentru a executa calcule sau operații logice, fără perioade lungi de inactivitate. În schimb, o aplicație care conține operații de tip yield sau wait are momente deliberate de pauză sau cedare a controlului, reducând utilizarea CPU-ului.

CB 2023 – Subiect rând impar

1. Oferiți exemple practice de comutatoare procesor memorie: discuție Sistem de Întrerupere, DMA și Canal I/E.

Sistemul de întrerupere constă în întreruperea procesorului de un dispozitiv extern (tastatură, mouse, rețea) pentru a procesa un eveniment urgent. Este trimis un semnal de întrerupere, se salvează contextul curent și se tratează întreruperea. Un exemplu concret este apăsarea unei taste. DMA permite transferul de date între dispozitivele periferice și memoria RAM fără intervenția constantă a procesorului. Un exemplu este transferul unui fișier pe un stick USB în RAM, unde DMA transferă direct blocurile date, iar procesorul este notificat doar la final, fiind liber pentru alte sarcini în timpul transferului. Canalul I/E este un procesor dedicat care gestionează comunicația cu dispozitivele periferice, autonom față de procesorul central. Un exemplu este reprezentat de gestionarea transferurilor cu imprimante, hard diskuri, rețea, fără ca procesorul să intervină în detaliile fiecărui transfer.

2. Descrieți comutatorul ierarhic de tip trunchi K.

Comutatorul trunchi K îmbină conectivitatea oferită de Cross-Bar-Switch cu costul redus al celorlalte tipuri de comutatoare. Într-o configurație cu M componente de tip A și N componente de tip B, orice componentă A poate comunica cu orice componentă B, însă nu întotdeauna vor fi min(M, N) comunicații simultane. În funcție de M, N și trafic, se va determina un număr de comunicații optim.

3. Comparați permutarea de bază (inverse shuffle) și permutarea cu intercalare perfectă (shuffle).

Permutarea de bază (numită și inverse shuffle) și permutarea cu intercalare perfectă (perfect shuffle) sunt două permutări elementare importante în arhitectura sistemelor de calcul, utilizate pentru conectarea comutatoarelor sau a altor resurse într-un sistem paralel. Principala diferență constă în permutarea biților: permutarea de bază face shift la dreapta, în timp ce permutarea cu intercalare perfectă face shift la stânga. Vedem, deci, că inverse shuffle inversează efectul intercalării perfecte. Tocmai de aceea sunt adesea considerate perechi inverse).

CC 2023

CB-CD 2024 – Subiect rând par

1. Descrieți folosind primitivele PMS componenta Comutator cu minim 5 primitive (e.g. atribute/valori)

Comutatorul are rolul de a face legătura între componentele unui sistem, asigurând legăturile dintre ele. Poate avea de la structuri foarte simple (buffer 3 state) până la complexe (comutare între procesoare și memorii). Poate fi caracterizat de următoarele atribute: structura ierarhică / neierarhică, sensul de comunicare simplex / duplex / semiduplex, numărul de legături realizate (numărul de intrări și de ieșiri) și concurența (câte comunicări concurente pot exista).

2. Comutatoare ierarhice: simplu, duplex și dual-duplex.

Comutatoarele ierarhice asigură comunicarea între componente de tip A și tip B. Comutatorul simplu conectează o componentă A la o componentă B, comutatorul duplex conectează o componentă A la mai multe componente de tip B, iar comutatorul dual-duplex conectează două componente A la mai multe componente de tip B.

3. Comparație arhitecturi CPU moderne cu acceleratoare GP-GPU.

CPU-urile sunt optimizate pentru execuția rapidă și flexibilă a unor sarcini variate, inclusiv secvențiale și cu ramificații complexe, având câteva nuclee puternice și cache-uri mari. Acceleratoarele GP-GPU sunt proiectate pentru paralelizare masivă, cu mii de nuclee simple care execută simultan multe thread-uri identice sau similare, fiind excelente pentru calcule vectoriale și date paralele.

4. Exemplificați modul de funcționare al DMA în sistemele cu procesoare de tip general CPU.

DMA permite transferul direct de date între dispozitivele periferice și memoria RAM, fără implicarea CPU, eliberând-ul pentru alte sarcini și crescând eficiența sistemului. CPU configurează controlul DMA prin adresa sursă, adresa destinație și dimensiunea transferului, după care controller-ul DMA preia controlul asupra magistralei de sistem și realizează transferul de date. La final trimite un interrupt către CPU când transferul este complet.

5. Cum e diferită o aplicație CPU intensivă de un yield sau wait (random)?

O aplicație CPU intensivă folosește în mod constant unitatea de procesare pentru a executa calcule sau operații logice, fără perioade lungi de inactivitate. În schimb, o aplicație care conține operații de tip yield sau wait are momente deliberate de pauză sau cedare a controlului, reducând utilizarea CPU-ului.

6. De ce nu este fezabilă realizarea unui procesor de 10TFLOPS?

Realizarea unui procesor de 10 TFLOPS (10 trilioane operații în virgulă mobilă pe secundă) nu este fezabilă în prezent din cauza mai multor limitări tehnologice și practice majore: consumul uriaș de energie și generarea excesivă de căldură care ar necesita soluții foarte complexe de răcire; constrângerile fizice legate de densitatea circuitelor și limitările tehnologice ale procesului de fabricație care împiedică miniaturizarea și integrarea unor unități de calcul suficient de numeroase; probleme legate de latențe și comunicații interne între componente, care afectează scalabilitatea frecvenței și paralelizarea eficientă; precum și costuri foarte mari de producție și proiectare. De aceea, în loc să se crească doar frecvența sau numărul de nuclee într-un singur procesor, soluțiile moderne folosesc arhitecturi paralele distribuite, multiple procesoare sau acceleratoare specializate pentru a atinge performanțe de ordinul zecilor de TFLOPS.

7. Cât din cod trebuie să fie paralelizabil pentru ca un GPU cu 4000 de core-uri să scaleze liniar?

Folosim legea lui Amdahl: $Speedup_{max} = \frac{1}{S+\frac{P}{N}}$, unde S = porțiunea secvențială din cod, P = porțiunea paralelizabilă, N = numărul de unități de procesare. Aplicând formula obținem $4000 = \frac{1}{(1-P)+\frac{P}{4000}} \rightarrow P = 100\%$, deci 100% din cod ar trebui să fie paralelizabil.

8. La ce sunt bune la ce nu sunt bune benchmark-urile? Oferiți exemple de benchmark-uri pentru diverse componente ale unui sistem de calcul diferite de LINPACK.

Benchmark-urile sunt tool-uri care măsoară performanța sistemelor de calcul, simulând comportamentul aplicațiilor reale. Ele nu folosesc, totuși, pentru a determina aspecte precum securitatea, disponibilitatea,

fiabilitatea sau scalabilitatea unui sistem de calcul. Exemple de benchmark-uri ar fi: SPEC pentru CPU sau HPCC pentru GPU.

9. Ce înseamnă active random scheduling și ce problemă încearcă să rezolve această tehnică?

Active random scheduling este o tehnică de planificare în care procesele sau thread-urile sunt selectate aleator, dar doar dintre cele care sunt active, adică gata să ruleze și care au resursele necesare disponibile. Spre deosebire de un scheduling pur aleatoriu care alege din toate procesele indiferent de stare, active random scheduling restrânge selecția la un subset relevant. Această tehnică încearcă să rezolve problema ineficienței și blocajelor care apar în sistemele cu mulți concurenți când procesele inactive sau blocate sunt alese pentru execuție, ceea ce ar risipi timp și resurse. Prin alegerea aleatorie doar dintre procesele active, scheduling-ul devine mai eficient, reducând timpul de așteptare și îmbunătățind utilizarea resurselor, păstrând totodată un nivel ridicat de echitate și simplificând implementarea.

10. Rețea de comutare de tip Delta: exemplu 8x27.

Presupunem că avem 8 procesoare și 27 de memorii. $8 = 2^3$ și $27 = 3^3$, deci folosim 3 nivele de CB 2x3. Dacă, de exemplu, vrem acces la memoria 6 (020 în baza 3), atunci se merge pe 0 (nivelul 0), 2 (nivelul 1) și 0 (nivelul 2).

CB-CD 2024 – Subiect rând impar